Document number	P2742R1
Date	2023-1-15
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

indirect dangling identification

Table of contents

Changelog

R1

- minor verbiage clarifications
- added the Tooling Opportunities section

Abstract

The `Bind Returned/Initialized Objects to the Lifetime of Parameters` proposal [1] floated providing a means for programmers to tell their compilers that a return is dependent upon some parameters such as `this`. The intent was to either `warn only about some possible buggy behavior` [1:1] or to `fix possible buggy behavior by extending the lifetime of temporaries` [1:2]. This path was not pursued because it would result in a viral attribution effort. This proposal tries to resume this path, eventually, with the intention of producing errors on clearly incorrect code.

Motivation

There are multiple resolutions to dangling in the `C++` language.

1. Produce an error
    - `Simpler implicit move` [2]
    - **This proposal**
2. Fix with block/variable scoping
    - `Fix the range-based for loop, Rev2` [3]
    - `Get Fix of Broken Range-based for Loop Finally Done` [4]
3. Fix by making the instance global

All are valid resolutions and individually are better than the others, given the scenario. This proposal is focused on the first option, which is to produce errors when code is clearly wrong.

This paper advocates adding the `parameter_dependency` attribute to the standard. This attribute can only be applied to functions both free and member. It can be applied more than once. It should be applied only once for each `dependent`. The `dependent` member takes a string which is the name of the parameter that is dependent upon other parameters. The `providers` member takes an array of strings which are the names of the parameters that the `dependent` parameter is dependent upon. A parameter can not be dependent upon oneself. The string “return” can be used for the return parameter and the string “this” can be used for the “this” pointer parameter.

```
[[parameter_dependency(dependent{"return"}, providers{"this", "left", "right", "fi
```

This new attribute may be used as in the following examples:

```
[[parameter_dependency(dependent{"return"}, providers{"i"})]]
int& h(bool b, int& i) {
  static int s;
  if (b) {
    return s;
  } else {
    return i;
  }
}
```

```
[[parameter_dependency(dependent{"return"}, providers{"left", "right"})]]
int& h(bool b, int& left, int& right) {
  if (b) {
    return left;
  } else {
    return right;
  }
}
```

```
class Car
{
private:
    Wheel wheels[4];
public:
    [[parameter_dependency(dependent{"return"}, providers{"this"})]]
    const Wheel& getDriverWheel() const {
        return wheels[0];
    }
}
```

This paper still has a viral attribution effort. So, why should we add this attribute to the standard and advocate for its use.

Documentation

Programmers could use the `parameter_dependency` attribute to document their libraries and to convey parameter dependency knowledge in a standard fashion to library users, even if creator and user is the same person. Without something like this, each programmer will continue to use verbiage that is different from other programmers. Currently, this type of documentation, if it even exists, may not even be in the header but in some other documentation. It is not uncommon for programmer’s to have to take apart someone else’s code in order to figure out how to use it safely.

Specification

The rationale is much the same as documentation except now it is means to standardize how we specify our proposals.

A Viral Attribution Effort

It should be noted that all functions do not need this attribute. Those that return references, reference types or pointers are desperately in need of this. Even if static analyzers such as that which is provided with `Microsoft Visual Studio` can do this analysis for warnings, this attribute or other similar one would still be needed at the public API boundary level of compiled libraries so that users of said libraries know how to use them.

Fixing indirect dangling by producing errors

Let's consider how this feature can help us fix even more dangling in `C++`.

> **C++ Core Guidelines**
> **F.43: Never (directly or indirectly) return a pointer or a reference to a local object** [5]
>
> ***Reason*** *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* [5:1]
>
> …
>
> ***Note*** *This applies only to non-static local variables. All static variables are (as their name indicates) statically allocated, so that pointers to them cannot dangle.* [5:2]

`Simpler implicit move` [2:1] is more than capable of identifying direct dangling of a local.

```
Car& f()
{
    Car local;
    return local;// error `Simpler implicit move` [^p2266r3]
}
```

What error can `C++` produce when `Car` local returns a tire?

```
const Wheel& f()
{
    Car local;
    return local.getDriverWheel();// error?
}
```

The fact is, there is no way of knowing with a compiled library without looking at the code and breaking `abstraction`. Even though the `Car` instance was local, `getDriverWheel` could be returning a global instance. However, the `parameter_dependency` attribute tells us, the

compiler and any static analyzers that the driver wheel's lifetime is dependent upon the car's instance which is a local and consequently this last example should have produced an error.

Tooling Opportunities

Compilers, static analyzers could further be enhanced to assert that the `parameter_dependency` matches what the documented function is doing.

Tooling could either error if a function in need hasn't been document, suggest the appropriate documentation or automatically add it.

None of these tooling opportunities are proposed at this time.

Summary

When coupled with `Simpler implicit move` [2:2] this proposal fixes even more dangling in the language, simply, especially of the more difficult to identify **indirect** types of dangling. It also serves as a standard way of doing documentation and specifications. This feature also works with pointers allowing its contribution back to the `C` programming language thus improving our entire ecosystem.

References

1. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf) ↩ ↩ ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html) ↩ ↩ ↩

3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf) ↩

4. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf) ↩

5. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object
   (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object) ↩ ↩ ↩